SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL :3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Constraint Based Problem

Code of Conduct:

I Pusalapati Chandu(192372119) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary

action.

Signature of the student: Pchandu

SIMATS ENGINEERING
ASSESSMENT TOOLS

1.Design a Pushdown Automaton that accepts the language
$L = \{ a^n b^n \mid n \geq 1 \}$
Constraint: You are allowed to use only one stack symbol (besides initial symbol)

2. CFG to PDA Conversion with Limited Productions
Given a Context-Free Grammar:
$S \rightarrow aSb \mid ab$
Construct an equivalent PDA.
Constraint: PDA must be constructed using only 2 states

3. PDA to CFG Conversion with Restricted Variables
Convert a given PDA into an equivalent CFG.
Constraint: The resulting CFG must use at most 3 variables (non-terminals)

4. Turing Machine with Limited Tape Movement
Design a Turing Machine to recognize the language
$L = \{ ww \mid w \in \{0,1\}^* \}$
Constraint: The tape head can move only to the right (no left movement allowed)

5. Consider a language $L = \{ a^n b^n c^n \mid n \geq 1 \}$
Constraint: Design a Turing Machine for L

SIMATS ENGINEERING
ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. PDA FOR L = { $a^n b^n$ | n ≥ 1 }

The question asks for a PDA accepting L = {$a^n b^n$ | n ≥ 1}, with only one stack symbol allowed besides the initial symbol.

Aim

To construct a PDA that accepts n a's followed by exactly n b's using only one working stack symbol X and the initial symbol Z0.

Algorithm

1. Start in q0 with Z0 on the stack.
2. For every a, push one X.
3. When the first b is read, move to q1 and pop one X.
4. In q1, pop one X for every b.
5. If an a appears after b processing starts, reject.
6. Accept when the input ends and only Z0 remains.

PDA Components

Component	Definition
States	q0: read a's; q1: read b's; qf: accept
Input alphabet	{a,b}
Stack alphabet	{Z0,X}
Initial state	q0
Acceptance	Input consumed with stack at Z0

Transition Table

State	Input	Top	Action	Next
q0	a	Z0	Push X	q0
q0	a	X	Push X	q0
q0	b	X	Pop X	q1
q1	b	X	Pop X	q1
q1	a	X	No transition	Reject
q1	end	Z0	Accept	qf

Flow

```
q0 -> read a -> push X
       repeat for every a
       | | first b
```

```
 v
q1 -> read b -> pop X
       repeat for every b
       |
       | input ends and stack = Z0
       v
     ACCEPT
```

Example

```
Input: aaabbb
a -> push X
a -> push X
a -> push X
b -> pop X
b -> pop X
b -> pop X
End -> Z0
Result -> ACCEPT
```

String	Reason	Result
ab	1 a, 1 b	Accepted
aabb	2 a, 2 b	Accepted
aaabbb	3 a, 3 b	Accepted
aab	Counts differ	Rejected
abb	Counts differ	Rejected
ba	Wrong order	Rejected

Justification

Each a adds exactly one X and each b removes exactly one X. Therefore the machine can finish successfully only when the numbers are equal. The state change at the first b ensures the required order a*b* is maintained. The constraint is satisfied because only X and Z0 are used as stack symbols.

Conclusion

The PDA recognizes $a^n b^n$ for $n \geq 1$ with one working stack symbol.

2. CFG TO PDA CONVERSION WITH ONLY 2 STATES

The given CFG is S → aSb | ab, and the PDA must use only two states. □filecite□turn8file0□L22-L26□

Aim

To construct an equivalent PDA using one processing state and one accepting state.

Grammar

```
S -> aSb
S -> ab
```

This grammar generates ab, aabb, aaabbb, and generally $a^n b^n$ for $n \geq 1$.

Construction Method

7. Push the start symbol S onto the stack.
8. In the processing state, replace S by aSb or ab nondeterministically.
9. If a terminal at the stack top matches the next input symbol, consume the input and pop the terminal.
10. When the input is completely consumed and the stack is empty except for Z0, enter the accepting state.

Two States

State	Purpose
q0	Grammar expansion and terminal matching
qf	Successful accepting state

Conceptual Transitions

Situation	Operation
S on stack	Replace with aSb or ab
a on stack and next input a	Consume a and pop
b on stack and next input b	Consume b and pop
Input finished + stack completed	q0 → qf

Example: aabb

```
S => aSb
  => a(ab)b
  => aabb

Stack:
Z0S
Z0aSb
```

```
Z0abb
then match a, a, b, b
then accept
```

Justification

The stack stores the grammar variable and pending terminals, so the states do not need to store the derivation history. One state can therefore perform all expansion and matching operations, with the second state reserved for acceptance.

Conclusion

The two-state restriction can be satisfied because the stack provides the memory required for grammar simulation.

3. PDA TO CFG CONVERSION WITH AT MOST 3 VARIABLES

The question asks for conversion of a given PDA into an equivalent CFG, with at most three non-terminals.

Important Note

The uploaded question does not contain the actual PDA's states, transition function, stack alphabet or acceptance condition. Therefore, an exact equivalent CFG cannot be derived from the source document alone.

Standard Method

A standard PDA-to-CFG construction uses variables that describe computations which remove a stack symbol while moving from one state to another.

```
[p A q]
```

The variable [p A q] represents strings that take the PDA from state p with A available on the stack to state q after the relevant stack symbol has been removed.

Steps

11. Put the PDA into a suitable standard form.
12. List its states and stack symbols.
13. Create state-to-state variables for stack-removal computations.
14. For each PDA transition, construct the corresponding CFG productions.
15. Create a start variable representing the PDA's initial configuration.
16. Remove unreachable variables and simplify the grammar.
17. Check whether the simplified grammar uses at most three variables.

Information needed for exact conversion	Available in uploaded file?
PDA states	No
Transitions	No
Stack alphabet	No
Initial configuration	No
Acceptance condition	No

Justification

The resulting CFG depends directly on the PDA transition function. Different PDAs produce different grammars, so inventing a PDA would not answer the assigned question accurately.

Conclusion

The formal conversion method is given, but the exact three-variable CFG requires the missing PDA diagram or transition table.

4. TURING MACHINE FOR L = { ww | w ∈ {0,1}* } WITH RIGHT-ONLY HEAD MOVEMENT

The question asks for a TM recognizing L = {ww | w ∈ {0,1}*}, while the tape head may move only right and never left.

Constraint Analysis

Under the standard one-tape Turing Machine model, the stated right-only restriction makes the requested general recognition impossible. To recognize ww, the machine must compare an arbitrarily long first half with the second half. A right-only head cannot revisit earlier cells.

Why

- The machine must determine or effectively compare the two halves.
- The first half can have arbitrary length.
- Finite control has only finitely many states and cannot store an arbitrary first half.
- A right-only tape head cannot return to earlier input cells.
- Therefore a standard one-tape right-moving-only TM cannot recognize arbitrary ww.

Example

For an input such as 0110, the machine needs to compare 01 with 10. For arbitrarily long inputs, the information from the first half is unbounded. Without left movement or another unbounded memory mechanism, that information cannot be retained in finite control.

Relaxation	What becomes possible
Allow left moves	Standard TM can mark and compare both halves.
Use multiple tapes / additional memory	First-half information can be stored and compared.
Right-only, one tape, standard finite control	General ww recognition is not possible.

Conclusion

The correct response under the exact constraint is an impossibility result for the standard one-tape TM model. A concrete TM can be designed if the right-only restriction is relaxed.

5. TURING MACHINE FOR L = { $a^n b^n c^n$ | n ≥ 1 }

The question asks for a Turing Machine for L = {$a^n b^n c^n$ | n ≥ 1}.

Aim

To recognize strings containing equal positive numbers of a, b and c in the order a*b*c*.

Tape Markers

Symbol	Meaning
a	Unprocessed a
b	Unprocessed b
c	Unprocessed c
X	Marked a
Y	Marked b
Z	Marked c
B	Blank

Algorithm

18. Check that the input has the order a+b+c+; otherwise reject.
19. Return to the left end.
20. Find the leftmost unmarked a and change it to X.
21. Move right and find the first unmarked b; change it to Y.
22. Move right and find the first unmarked c; change it to Z.
23. Return to the left end.
24. Repeat until there is no unmarked a.
25. Scan for any remaining unmarked b or c.
26. If none remain, accept; otherwise reject.

State Flow

```
q0: check input order
 |
 v
q1: find unmarked a -> X
 |
 v
q2: find unmarked b -> Y
 |
 v
q3: find unmarked c -> Z
 |
 v
q4: return to left
 |
 +-- unmarked a? -- yes --> q1
 |
 no
 v
q5: check remaining b/c
 |
 +-- found --> REJECT
 |
 none
 v
ACCEPT
```

Example: aaabbbccc

```
Round 1: XaaYbbZcc
Round 2: XXaYYbZZc
Round 3: XXXYYYZZZ
No unmarked a, b or c
Result: ACCEPT
```

Rejected Example: aabbbcc

```
Round 1: X a Y bb Z c
Round 2: X X Y Y b Z Z
Unmarked b remains
Result: REJECT
```

Correctness

If the machine accepts

Every a was matched with exactly one b and one c, and the input order was checked. Therefore the string has equal numbers of a, b and c and is in $a^n b^n c^n$.

If the input is in the language

For $a^n b^n c^n$, each round finds one remaining a, one b and one c. After n rounds, all symbols are marked, so the machine accepts.

Conclusion

The marking-and-rescanning TM recognizes exactly $a^n b^n c^n$ for $n \geq 1$.

FINAL ANSWER SUMMARY

Q	Topic	Result
1	PDA for $a^n b^n$	Possible using one working stack symbol X.
2	CFG → PDA	Possible using two states.
3	PDA → CFG	Exact answer requires the missing PDA; standard method provided.
4	TM for ww, right-only	Impossible for a standard one-tape right-only TM.
5	TM for $a^n b^n c^n$	Possible using repeated marking and matching.